

NEXO

Aplicación de IA que propone hipótesis científicas conectando
literatura de áreas que no se citan entre sí

Aplicación en vivo	◀ COMPLETAR TRAS EL DEPLOY EN share.streamlit.io ▶
Repositorio (código, datos e informe)	https://github.com/eliasjz36/nexo
Registro de desarrollo (co-work con IA)	https://github.com/eliasjz36/nexo/blob/main/DEVLOG.md

Resumen del resultado: el sistema leyó únicamente artículos científicos anteriores a 2013 de dos áreas que no se citaban entre sí (farmacología de inhibidores SGLT2 e insuficiencia cardíaca) y ubicó la conexión entre ambas —que la medicina confirmó recién en 2015-2019— en el puesto 13 de 772 hipótesis generadas. Al momento del corte no existía ningún artículo que mencionara ambos temas juntos; hoy hay 225.

Alumno: Elías Jiménez
Fecha: Agosto de 2026

Parte 1 — De la idea conceptual a la aplicación real

1.1 Qué se construyó

En el trabajo de medio ciclo, NEXO era un diseño conceptual: un sistema de agentes que leería literatura científica, armaría una base de conocimiento y propondría hipótesis conectando áreas separadas, con una memoria persistente y un ciclo de mejora. En esta entrega ese diseño existe como software funcionando: un pipeline de cinco pasos que corre en la máquina del alumno con un LLM local, y una aplicación web pública que muestra los resultados.

Para poder evaluar el sistema con honestidad se eligió un experimento con corte temporal: el corpus solo contiene artículos anteriores a 2013 sobre dos áreas que entonces no se citaban entre sí — la farmacología de los inhibidores SGLT2 (un fármaco para la diabetes) y la fisiopatología de la insuficiencia cardíaca. La conexión real entre ambas se publicó recién a partir de 2015. Si el sistema la encuentra sin haberla podido leer, el método funciona. Es la forma estándar de evaluar sistemas de descubrimiento basado en literatura (time-slicing).

- **Corpus:** 1.987 artículos de PubMed con resumen y términos MeSH (865 del área SGLT2, 1.122 de insuficiencia cardíaca), con corte estricto al 31/12/2013. Solo 7 artículos mencionaban ambos temas antes del corte y fueron excluidos.
- **Extracción:** 11.182 relaciones tipadas y con dirección (aumenta, reduce, causa, previene, trata, inhibe, se asocia, no afecta), cada una con la frase textual del artículo que la respalda. Todo con un LLM local (sección Parte 2).
- **Verificación de evidencia:** una capa de integridad descartó el 17% de las relaciones porque su frase de respaldo no existía en el artículo o sus entidades no aparecían en el texto (ver 1.8, riesgo R2). El grafo final tiene 9.271 relaciones verificables.
- **Hipótesis:** 772 pares (A, B) que nunca aparecen juntos en el corpus, conectados por conceptos puente con signos coherentes, ordenados por convergencia de puentes independientes.
- **Agentes:** captura, extractor, normalizador, descubridor, verificador de novedad (consulta a PubMed) y escéptico (intenta refutar cada hipótesis top). La memoria persistente es la base SQLite que viaja en el repositorio.

1.2 Resultado del experimento

Con el corpus congelado en 2013, el sistema generó 772 hipótesis. La conexión que la medicina confirmó después aparece así:

Verificación	Valor
--------------	-------

Hipótesis «sglt2 inhibition = heart failure» (sentido opuesto = posible beneficio)	Puesto 13 de 772
Puentes mecánicos encontrados	hipertensión y diabetes (ambos con signos coherentes)
Artículos en PubMed que mencionaban ambos términos hasta 2013	0
Artículos que los mencionan hoy (2026)	225
Veredicto del agente escéptico	«dudosa» (ver análisis abajo)

Además, las tres primeras hipótesis del ranking dicen que la inhibición SGLT2 actúa en el mismo sentido que captopril, furosemida y tolvaptán — exactamente los fármacos con que se trata la insuficiencia cardíaca. Es la misma conclusión por otro camino: el sistema no sabía que ese fármaco de diabetes servía para el corazón, pero encontró que se comporta como los que sí sirven.

El escéptico calificó la hipótesis principal como «dudosa»: aceptó los puentes de hipertensión y diabetes pero atacó un tercer puente (natriuresis) que venía con un signo mal extraído, usando una contradicción del propio grafo. Es el comportamiento buscado: con la evidencia disponible en 2013, un revisor riguroso habría dicho exactamente eso. La decisión final es siempre del experto humano.

Limitación declarada: el caso de estudio fue elegido a propósito donde se sabía que existía una conexión confirmada después del corte (práctica estándar del área para poder medir). El sistema también genera cientos de hipótesis sin confirmación conocida; distinguir cuáles valen la pena sigue dependiendo del experto.

1.3 Diagrama de arquitectura

Arquitectura de NEXO (como quedó construida)

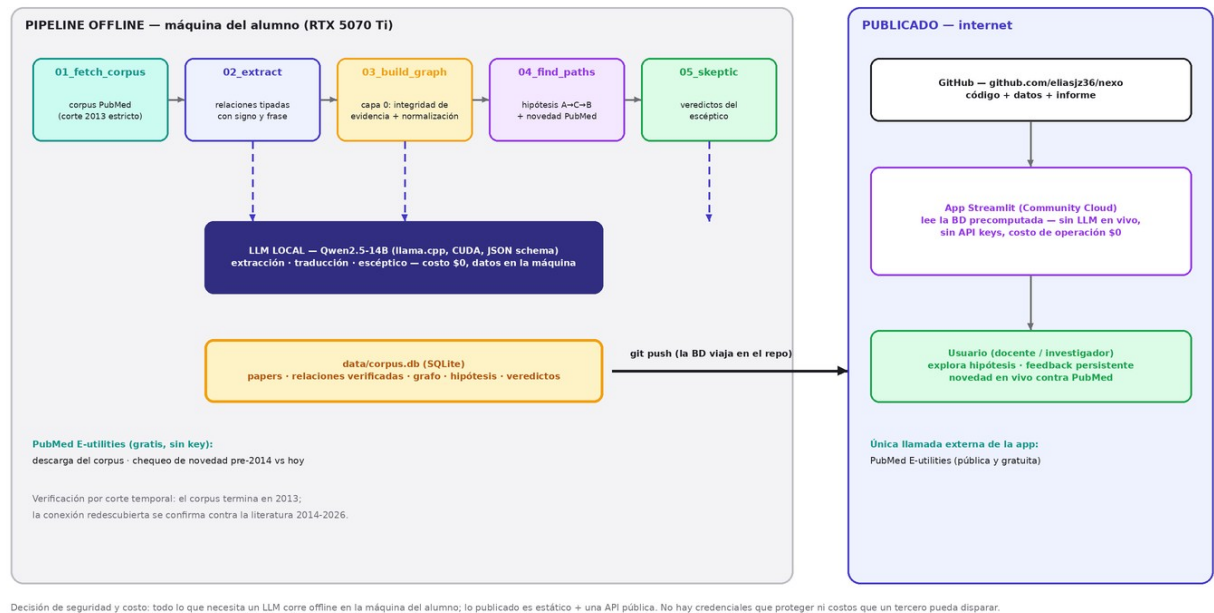


Figura 1. Arquitectura construida: pipeline offline con LLM local en la máquina del alumno; lo publicado no necesita claves ni genera costos.

La decisión central de arquitectura: todo lo que requiere un LLM corre offline y sus resultados quedan en una base SQLite que se versiona en el repositorio. La aplicación publicada solo lee esa base; su única llamada externa es a la API pública y gratuita de PubMed para re-chequear la novedad. Así la app funciona sin claves, sin costos de operación y sin depender de que el LLM esté disponible.

1.4 Diagrama UML (secuencia)

Diagrama UML de secuencia — consulta de una hipótesis



Figura 2. Secuencia de una consulta: la app resuelve con lecturas locales, registra la sesión y persiste el feedback del usuario (memoria).

1.5 Tecnologías utilizadas

Tecnología	Para qué	Por qué se eligió
Python 3.12	Todo el pipeline y la app	Lenguaje del ecosistema de datos; ya instalado.
SQLite	Corpus, grafo, hipótesis, feedback y logs	Un solo archivo versionable en git; sin servidor de base de datos que administrar ni exponer.
llama.cpp + Qwen2.5-14B (Q4_K_M)	Extracción, traducción y escéptico	LLM local en la GPU propia (RTX 5070 Ti): costo cero y datos que no salen de la máquina. Se recompiló con CUDA 13.1.
Salida forzada por JSON Schema	Todas las llamadas al LLM	El modelo no puede responder fuera del formato; elimina el parseo frágil de texto libre.
PubMed E-utilities	Descarga del corpus y chequeo de novedad	API oficial, gratuita y sin clave; incluye los términos MeSH usados para normalizar.
Streamlit + Community Cloud	Interfaz web publicada	Framework simple en Python; hosting gratuito con deploy

		directo desde GitHub.
GitHub	Código, datos, informe y despliegue	Repositorio público exigido por la consigna; origen del deploy.
AppTest (Streamlit)	Prueba automatizada de la app	Ejecuta la app completa sin navegador; detectó un error real antes del deploy.

1.6 Capturas de la aplicación

El experimento

El sistema leyó solo papers anteriores a 2013 de dos áreas que no se citaban entre sí:

- farmacología de inhibidores SGLT2 (diabetes)
- insuficiencia cardíaca (manejo de sodio y volumen)

La conexión real entre ambas la confirmó la ciencia recién en 2015-2019. Si aparece acá, el método funciona.

Papers leídos

1987

Relaciones

9271

Conceptos

6172

Hipótesis

772

☐ Solo las que se oponen al blanco (posible beneficio)

☐ Solo las que superaron al escéptico

NEXO

Hipótesis científicas a partir de conexiones no exploradas en la literatura

⚠️ Prototipo académico. Las hipótesis que se muestran son candidatos generados automáticamente a partir de literatura científica: no son consejo médico y requieren validación de expertos antes de cualquier uso.

Hipótesis (50)

❌ sgl2 inhibition → captopril · 3 puente(s)

👉 sgl2 inhibition → furosemide · 3 puente(s)

👉 empagliflozin → captopril · 4 puente(s)

👉 canagliflozin → captopril · 3 puente(s)

👉 sgl2 inhibition → prednisone · 2 puente(s)

❌ sgl2 inhibition → tolvaptan · 2 puente(s)

👉 dapagliflozin → captopril · 2 puente(s)

👉 sgl2 inhibition → adrenomedullin · 2 puente(s)

👉 sgl2 inhibition → renal artery revascularisation · 2 puente(s)

👉 dapagliflozin → furosemide · 2 puente(s)

👉 sgl2 inhibition → clonidine · 2 puente(s)

👉 sgl2 inhibition → hemofiltration · 2 puente(s)

👉 ouabain → alpha-hanp · 3 puente(s)

👉 diabetes mellitus → diuretics · 1 puente(s)

👉 sgl2 inhibition → ultrafiltration · 2 puente(s)

👉 sgl2 inhibition → heart failure · 2 puente(s)

👉 sgl2 inhibition → ventricular pacing · 2 puente(s)

👉 dapagliflozin → renal artery revascularisation · 2 puente(s)

❌ phlorizin → prednisone · 2 puente(s)

👉 canagliflozin → furosemide · 2 puente(s)

👉 ouabain → furosemide · 2 puente(s)

👉 sgl2 inhibition → natriuretic peptides · 2 puente(s)

👉 phlorizin → captopril · 2 puente(s)

👉 ouabain → atrial natriuretic peptide · 2 puente(s)

👉 ouabain → adrenomedullin · 2 puente(s)

sglt2 inhibition → captopril

Hipótesis: sgl2 inhibition actuaría en el mismo sentido que captopril — si el blanco es un fármaco, sugiere un efecto similar; si es una patología, un posible riesgo. Los dos conceptos nunca aparecen juntos en el corpus.

Novedad

Papers juntos pre-20140

Papers juntos hoy1

Re-chequear novedad en PubMed (en vivo)

Veredicto del escéptico

❌ REFUTADA — Se propone que la inhibición de sgl2 y captopril tienen efectos similares en el peso corporal y la presión arterial sistólica, pero la evidencia muestra contradicciones y efectos opuestos en natriuresis.

A favor: La evidencia sugiere que tanto la inhibición de sgl2 como captopril pueden reducir la presión arterial y el peso corporal.

En contra: Hay contradicciones significativas: la inhibición de sgl2 no afecta el peso corporal en algunos estudios, mientras que captopril no induce natriuresis en otros. Además, la natriuresis es aumentada por captopril pero no por la inhibición de sgl2.

Figura 3. Vista general: métricas del experimento, filtros y lista de hipótesis con el ícono del veredicto del escéptico.

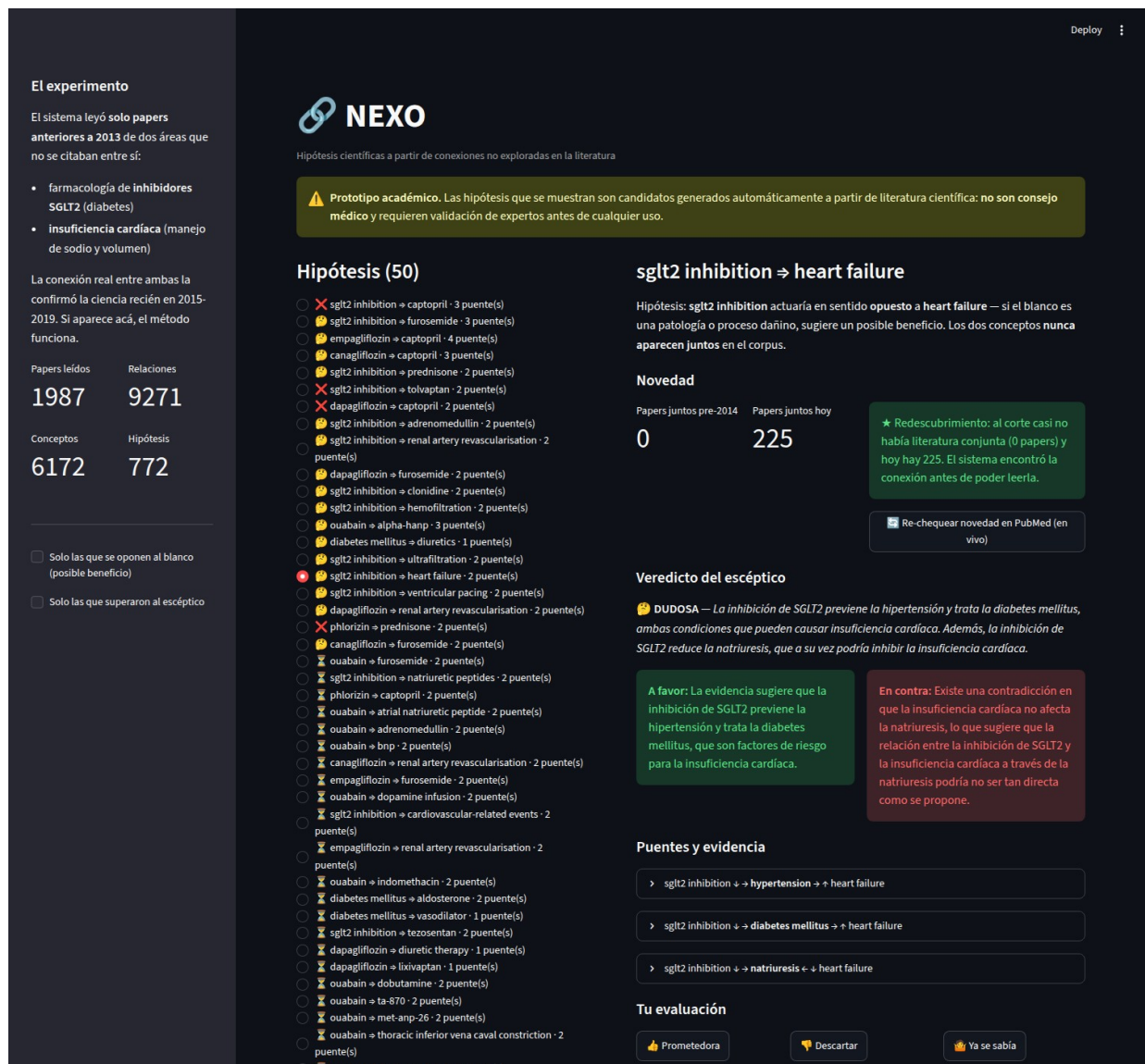


Figura 4. Detalle de la hipótesis redescubierta: novedad (0 artículos pre-2014, 225 hoy), veredicto del escéptico con argumentos a favor y en contra, y puentes con su evidencia trazable (enlaces a PubMed).

1.7 Registro de una sesión real de uso

La aplicación registra automáticamente cada interacción en la tabla registro_uso de la base (es, además, la evidencia de uso que pide esta sección). Extracto real:

REGISTRO DE SESIÓN DE USO — tabla registro_uso de la app
(cada interacción del usuario queda registrada automáticamente)

```
=====
2026-08-15T18:04:56 ver:hipotesis=1:sglt2 inhibition=>nesiritide
2026-08-15T18:04:56 ver:hipotesis=1:sglt2 inhibition=>nesiritide
2026-08-15T18:05:21 ver:hipotesis=1:sglt2 inhibition=>nesiritide
2026-08-15T18:05:21 ver:hipotesis=1:sglt2 inhibition=>nesiritide
2026-08-15T19:15:44 ver:hipotesis=434:sglt2 inhibition=>captopril
```


2026-08-15T19:15:44 ver:hipotesis=434:sglt2 inhibition=>captopril
2026-08-16T19:24:08 ver:hipotesis=434:sglt2 inhibition=>captopril
2026-08-16T19:24:09 ver:hipotesis=451:sglt2 inhibition=>heart failure
2026-08-16T19:24:47 ver:hipotesis=434:sglt2 inhibition=>captopril

Las entradas «ver:» corresponden a la apertura del detalle de una hipótesis; también se registran los re-chequeos de novedad y el feedback (prometedora / descartada / conocida), que queda persistido y no vuelve a preguntar en visitas futuras.

1.8 Autoevaluación UX/UI — heurísticas de Nielsen

Público objetivo: un investigador o docente que quiere explorar hipótesis y juzgar su evidencia. Evaluación honesta de la interfaz actual:

Heurística	Evaluación
1. Visibilidad del estado	Bien: métricas siempre visibles; spinners de Streamlit durante cargas; el feedback muestra la marca guardada.
2. Coincidencia con el mundo real	Parcial: la interfaz está en español pero las entidades científicas quedan en inglés (decisión de normalización con MeSH). Para el público objetivo —que lee papers en inglés— es aceptable; para un público general no lo sería.
3. Control y libertad del usuario	Parcial: el feedback se puede cambiar (sobrescribe), pero no hay un «deshacer» explícito ni forma de limpiar filtros de un clic.
4. Consistencia y estándares	Bien: iconografía uniforme (✅😞❌), mismo patrón de tarjetas y expansores en toda la app.
5. Prevención de errores	Bien: no hay entradas de texto libre; todo es selección. El re-chequeo de novedad maneja el error de red con un mensaje.
6. Reconocimiento antes que recuerdo	Bien: cada hipótesis muestra su cadena completa y su evidencia; no hay códigos que memorizar.
7. Flexibilidad y eficiencia	Bien: enlaces directos ?hip=id para compartir una hipótesis; filtros para usuarios avanzados.
8. Diseño minimalista	Bien: dos filtros, una lista y un detalle; sin funciones decorativas.
9. Ayudar a reconocer y recuperarse de errores	Parcial: los errores de red se informan, pero sin sugerencia de causa.
10. Ayuda y documentación	Débil: la explicación del experimento está en la barra lateral, pero falta una sección de ayuda sobre cómo interpretar signos y puentes. Es la

	mejora pendiente más clara.
--	-----------------------------

1.9 Log de ciberseguridad

Riesgos identificados durante el desarrollo y medidas tomadas:

#	Riesgo	Medida tomada
R1	Entrada no confiable al LLM: los abstracts son texto externo que se inserta en el prompt (inyección indirecta).	El LLM corre local, sin herramientas ni acceso a nada (solo devuelve JSON); la salida está forzada por esquema; y ninguna salida se ejecuta: solo se almacena como datos.
R2	Alucinación del extractor: el modelo inyectó conocimiento de su pre-entrenamiento como si estuviera en el texto (llegó a «extraer» que SGLT2 trata la insuficiencia cardíaca de papers de química que no la mencionan).	Capa de integridad de evidencia: la frase de respaldo debe existir textualmente en el abstract y las entidades deben estar ancladas al texto. Se descartó el 17% de las relaciones. Sin esta capa, el experimento de corte temporal quedaba contaminado con el «futuro».
R3	Exposición de claves o costos disparados por abuso de la app pública.	La arquitectura elimina el riesgo: no hay ninguna clave (el LLM es offline y la única API en vivo es pública y gratuita). No hay nada que robar ni costo que disparar.
R4	Uso indebido de las hipótesis como consejo médico.	Aviso permanente y visible en la app; el sistema nunca recomienda acciones clínicas; el flujo exige el juicio del experto (feedback humano) y el escéptico expone los argumentos en contra.
R5	Cadena de dependencias y datos.	Solo dos dependencias directas (requests, streamlit); datos públicos de PubMed sin información personal; base versionada en el repositorio con historial auditable.

1.10 Cómo se usó la IA en el desarrollo (co-work)

El desarrollo completo se hizo en pareja con un asistente de IA (Claude Code), y el registro detallado quedó en el DEVLOG.md del repositorio, escrito a medida que pasaban las cosas. Lo esencial:

- **Qué hizo la IA:** el análisis de factibilidad previo (verificando APIs, costos y disponibilidad de datos contra fuentes reales), la escritura de la mayor parte del código, los diagramas, y la ejecución y depuración del pipeline. El alumno definió el problema, tomó las decisiones de alcance y validó los resultados.
- **Dónde falló / qué costó:** (1) el filtro de fechas de PubMed dejó pasar 31 papers de 2014 y hubo que filtrar por año contra el dato descargado; (2) el build local de llama.cpp era solo CPU y hubo que recompilar con CUDA; (3) el puerto 8080 estaba ocupado por otro servicio local; (4) la más importante: el extractor alucinó relaciones con conocimiento posterior al corte, y hubo que diseñar la capa de verificación de evidencia (riesgo R2).
- **Qué sorprendió:** que la falla más grave no la anticipó ningún plan: se encontró inspeccionando datos. Y que el modelo llegó a citar frases reales del abstract atribuyéndoles entidades que el paper nunca menciona — un modo de error más fino que la alucinación evidente, que obligó a una segunda capa de verificación.

Parte 2 — LLM local en el proyecto

La consigna pide una reflexión sobre cómo se integraría un LLM o SLM local. En este proyecto la integración no es hipotética: el LLM local es el motor del sistema. Esta sección describe ese rol y lo aprendido, con los números reales medidos.

2.1 Rol en la arquitectura

Un Qwen2.5-14B-Instruct cuantizado (Q4_K_M, ~9 GB), servido con llama.cpp sobre la GPU del alumno (RTX 5070 Ti, 16 GB), cumple tres funciones del pipeline: extraer las relaciones tipadas de cada abstract, traducir a inglés canónico las entidades que salen mal, y actuar como agente escéptico sobre las hipótesis principales. Todas las llamadas usan salida forzada por JSON Schema, así el modelo no puede responder fuera del formato.

2.2 Qué le aporta al usuario final

- La aplicación publicada es gratuita y no puede quedarse sin crédito, porque el procesamiento pesado ya ocurrió offline y costó \$0.
- Privacidad por diseño: ningún texto pasa por servicios de terceros durante el procesamiento (relevante si el corpus fuera privado, p. ej. documentos internos de un laboratorio).
- Reproducibilidad: cualquiera con una GPU similar puede correr el pipeline completo del repositorio sin contratar nada.

2.3 Qué aprendimos como profesionales

Medición real	Valor
Velocidad de extracción (4 pedidos en paralelo)	≈1,9 s por abstract
Corpus completo (1.987 abstracts)	≈1 hora de GPU
Costo total de LLM del proyecto	\$0
Relaciones descartadas por evidencia no verificable	17%
VRAM utilizada (modelo + contexto)	≈13 GB

La lección central: con un modelo local de 14B la calidad no viene garantizada — viene con un contrato distinto. El modelo extrae rápido y gratis, pero alucina con la suficiente frecuencia como para que una capa de verificación automática

contra la fuente sea obligatoria, no opcional. El costo que no se paga en dólares se paga en ingeniería de validación.

2.4 Limitaciones reales frente a una API en la nube

- **Calidad:** un modelo frontera comete menos errores de extracción; con el local hicieron falta dos iteraciones de prompt y dos capas de filtrado que con un modelo mayor quizá serían menores (aunque no nulas).
- **Hardware:** exige una GPU con VRAM suficiente; el primer intento con el build de CPU habría tardado 20-30 horas en lugar de una.
- **Operación:** compilación con CUDA, gestión de puertos y memoria, y mantenimiento propio; con una API eso lo opera el proveedor.
- **Escala:** una API en la nube ofrece procesamiento por lotes masivo y modelos más capaces al costo de depender de un tercero, exponer datos y pagar por token.

2.5 Demostración: Ollama corriendo el modelo localmente

Se instaló Ollama (modo usuario, desde el paquete oficial) y se importó el mismo modelo del pipeline desde el archivo GGUF local mediante un Modelfile, sin volver a descargarlo. Transcripción real — nótese en «ollama ps» que el modelo corre 100% en GPU:

```
=====
DEMO OPCIONAL: Ollama corriendo localmente (RTX 5070 Ti)
=====
```

```
$ ollama --version
ollama version is 0.15.5-rc0
```

```
$ ollama list
NAME                ID                SIZE   MODIFIED
qwen2.5-14b-local:latest  fefeb9b19068  9.0 GB  14 seconds ago
```

PREGUNTA (vía /api/chat): En el patrón A-C-B que usa NEXO: si un fármaco aumenta la natriuresis (C) y la insuficiencia cardíaca cursa con retención de sodio, ¿qué hipótesis se puede plantear y qué habría que verificar antes de tomarla en serio? Respondé en 4-5 líneas.

RESPUESTA:

Si un fármaco aumenta la natriuresis (C) en pacientes con insuficiencia cardíaca que sufren retención de sodio, se podría plantear la hipótesis de que este tratamiento podría ayudar a reducir la carga hídrica y mejorar el estado clínico del paciente. Sin embargo, antes de tomar esta hipótesis en serio, sería necesario verificar si efectivamente hay una mejora clínica significativa sin provocar desequilibrios eletrolíticos adversos ni empeorar otros síntomas cardíacos.

```
$ ollama ps
```

NAME	ID	SIZE	PROCESSOR	CONTEXT	UNTIL
qwen2.5-14b-local:latest	e528d44823af	9.7 GB	100% GPU	4096	4 minutes from now

Detalle técnico honesto: al importar el GGUF crudo, Ollama no aplicó la plantilla de chat del modelo y las respuestas divagaban sin cortar; hubo que declararla en el Modelfile (formato ChatML de Qwen con sus tokens de parada). En el pipeline, servido con llama.cpp, el mismo archivo funcionó directo. Es el tipo de fricción operativa que implica administrar un LLM propio (sección 2.4).